第3組期末報告

GREENFLOW
綠流

A1105507 蘇柏諺
A1105508 陳建穎
A1105521 黎子崴

關於綠流GREENFLOW

綠流 GreenFlow 系統是一個智慧垃圾分類系統，結合影像辨識技術與深度學習模型ResNet50，即時對垃圾進行準確分類，分類結果即時顯示於 OLED 螢幕上。系統採模組化設計，具備高效分類、支援多硬體平台及低部署成本特性，可靈活整合至各類回收設備，推動垃圾分類智慧化與自動化，提升資源回收效率，減少環境污染，幫助永續發展目標的實現。

目錄

- 專題簡介
- 使用平台
- 應用之技術及知識
- 問題與解決
- 作品展示
- 工作分配

專題簡介

系統架構

機器學習

資料集
利用 Kaggle 的 data set 作為垃圾辨識的訓練集

模型構建與訓練
利用Pytorch做數據預處理，並建構模型與訓練

影像處理

物件偵測
使用 ResNet50 實現即時物件偵測，準確定位攝影機畫面中的垃圾位置

輸出結果



即時反饋
視覺化輸出幫助用戶理解系統判斷

功能說明

使用者

使用者放置垃圾

GREENFLOW

分辨該垃圾的分類

最終

精準判斷該垃圾所屬之分類

系統特色

即時垃圾分類	利用影像辨識技術，快速準確地對垃圾進行分類，支援多種垃圾類別，並即時顯示分類結果於 OLED 螢幕上。
高準確率分類	基於 ResNet50 深度學習模型，經由特定垃圾數據集的訓練與優化，分類準確率超過 95%。
模組化設計	採用模組化設計，支援多種硬體平台（如 Jetson Nano），能靈活接入攝影機、顯示模組。

系統特色

市場性

根據市場分析，2024 年全球垃圾分類機器人市場規模預計達到 21.3 億美元，各國亦將垃圾分類列為重要政策推動方向。綠流 GreenFlow 系統具備廣泛的市場應用潛力，能進一步支援智慧回收機器與垃圾分類基礎設施建設，成為未來智慧城市的一部分。

延伸性

透過持續的模型訓練與數據優化，系統將具備辨識更多垃圾類型的能力，提升分類精準度，並可根據不同地區的垃圾分類需求靈活調整，提供長期可持續的服務。

SDGs 關鍵指標

目標 11: 永續城市與社區

提高垃圾分類效率：
透過人工智慧進行準確分類，減少人工錯誤。

減少垃圾污染：
自動化垃圾分類能避免混合垃圾導致的環境污染，提升城市清潔。

目標 12: 責任消費與生產

推動資源循環：
自動將可回收垃圾（如紙類、塑膠）與其他垃圾分開。

教育與引導：
在系統中顯示分類結果，幫助用戶增強垃圾分類的環保意識。

使用平台
&硬體設備

硬體平台

Jetson Nano

作為核心運算平台，支持深度學習模型的推論和影像處理。

攝影機

用於即時影像捕捉，提供垃圾物件的清晰畫面作為模型輸入。

OLED 顯示螢幕

即時輸出分類結果，提供清晰的用戶交互體驗。

軟體平台

Kaggle

提供垃圾分類的數據集作為模型訓練資料來源。

PyTorch

作為深度學習框架，用於模型構建、訓練、測試與推論。

應用的
技術及知識

技術

機器學習與深度學習技術

資料集選擇與處理	用 Kaggle 平台上的垃圾分類資料集作為訓練資料
模型構建與訓練	基於 ResNet50 深度學習模型進行垃圾分類
深度學習框架	PyTorch ： 用於數據讀取、模型構建、訓練與測試。

技術

影像辨識與即時處理技術

ResNet50 作為核心分類模型，實現垃圾物件的偵測與分類，並優化推論速度以滿足即時性需求。

對攝影畫面中的垃圾物件
進行準確定位與處理。

知識

聯合國永續發展目標應用

目標 11：永續城市與社區

使用自動化垃圾分類技術，減少環境污染，提高垃圾分類效率

目標 12：責任消費與生產

推動資源循環與再利用，提升垃圾回收率。

問題與解決

問題01

Python 與 TensorRT 版本衝突
python 需要搭配對應版本的 TensorRT 。

Adafruit 版本衝突
- 原本使用 Adafruit 套件與 Python 版本不符或衝突。

1. 經過多次的版本調整才解決TernsorRT的版本問題。

2.改用 luma.oled： 與 Adafruit 的原套件衝突，改用 luma.oled 才解決OLED顯示問題。

問題02

需要使用獨立顯卡
- 因資料量大、訓練次數（epoch）多，且需更高的運算效能來提升訓練速度與最終精度（acc）。

借助他人顯卡並反覆嘗試
- 透過同學的顯卡跑訓練流程，針對超參數（batch size、learning rate 等）做 try & error。

作品展示

程式碼展示（1）

```python
from torchvision.datasets import ImageFolder
import torchvision.transforms as transforms

transformations = transforms.Compose([
    transforms.Resize((256, 256)),                  # 調整大小
    transforms.ToTensor(),                          # 轉為 Tensor
    transforms.Normalize(mean=[0.485, 0.456, 0.406],  # 標準化（以 ImageNet 預訓練模型
                         std=[0.229, 0.224, 0.225])
])

dataset = ImageFolder(data_dir, transform = transformations)
```

```python
from torch.utils.data import random_split

# 計算資料集的大小
dataset_size = len(dataset)  # 獲取資料集總樣本數
train_size = int(0.7 * dataset_size)  # 訓練集 70%
val_size = int(0.2 * dataset_size)    # 驗證集 20%
test_size = dataset_size - train_size - val_size  # 測試集 10%

# 分割資料集
train_ds, val_ds, test_ds = random_split(dataset, [train_size, val_size, test_size])
```

準確率

```
Epoch 1: train_loss: 1.4459, val_loss: 1.2375, val_acc: 0.8296
Epoch 2: train_loss: 1.1654, val_loss: 1.1503, val_acc: 0.9175
Epoch 3: train_loss: 1.0987, val_loss: 1.1303, val_acc: 0.9248
Epoch 4: train_loss: 1.0725, val_loss: 1.1186, val_acc: 0.9409
Epoch 5: train_loss: 1.0661, val_loss: 1.1194, val_acc: 0.9253
Epoch 6: train_loss: 1.0608, val_loss: 1.1130, val_acc: 0.9312
Epoch 7: train_loss: 1.0621, val_loss: 1.1234, val_acc: 0.9275
Epoch 8: train_loss: 1.0625, val_loss: 1.1130, val_acc: 0.9307
模型權重已保存為 model_weights.pt
```

程式碼展示（2）

```
class ResNet(ImageClassificationBase):
    def __init__(self):
        super().__init__()
        # Use a pretrained model
        self.network = models.resnet50(pretrained=True)
        # Replace last layer
        num_ftrs = self.network.fc.in_features
        self.network.fc = nn.Linear(num_ftrs, len(dataset.classes))

    def forward(self, xb):
        return torch.sigmoid(self.network(xb))

model = ResNet()
```

程式碼展示（3）

```
def do_inference(context, inputs, outputs, bindings, stream):
    [cuda.memcpy_htod_async(inp["device_mem"], inp["host_mem"], stream) for inp in i
    context.execute_async_v2(bindings=bindings, stream_handle=stream.handle)
    [cuda.memcpy_dtoh_async(out["host_mem"], out["device_mem"], stream) for out in o
    stream.synchronize()
    return outputs[0]["host_mem"]

# 顯示分類結果在 OLED 上
Codeium: Refactor | Explain | Generate Docstring | ×
def display_oled_info(fps, label, confidence):
    image = Image.new("1", (oled.width, oled.height))
    draw = ImageDraw.Draw(image)
    draw.rectangle((0, 0, oled.width, oled.height), outline=0, fill=0)
    draw.text((0, 0), f"FPS: {fps:.2f}", font=font, fill=255)
    draw.text((0, 20), f"Label: {label}", font=font, fill=255)
    draw.text((0, 40), f"Conf: {confidence:.2f}", font=font, fill=255)
    oled.display(image)
```

成果展示


Garbage Classifier
Class: cardboard


FPS: 16.73
Label: cardboard
Conf: 0.21

成果展示


Garbage Classifier
Class: metal
環保去
CLE
CLEANING
(NON-ARO
規格或等級
us pickle data which will execute


VCC GND SCL SDA
FPS: 16.64
Label: metal
Conf: 0.22

成果展示


Class: plastic
pickle data which will execute arbitrary code during unpickling (

工作分配

製作時數：約15小時

蘇柏諺 A1105507	陳建穎 A1105508	黎子崴 A1105521
• 訓練模型 • 製作簡報 • 上台簡報	• 訓練模型 • 製作簡報 • 上台簡報	• 訓練模型 • 製作簡報(主要) • OLED 顯示模組 • 錄製參賽影片

感謝聆聽